

SEIS 606: Data Analytics and Visualization
Fall 2026
University of St. Thomas

Project 1 Requirements

Submitted by 5:15 PM Wednesday, October 21st and presented in class.

Purpose

Build an interactive app with AI assistance, test it deliberately, and show what you learned. It doesn't need to be perfect. It needs to run, and you need to understand where it falls short and what it would take to fix it.

What you hand in

1. **The app**, in a repo I can open, with the spec and the Spec Kit artifacts (spec, plan, tasks) included.
2. **A test log** (see below).
3. **A brief write-up** of 1-2 pages covering the analysis section below.
4. **A 5-7 minute presentation** with a live demo, leaving time for questions/discussion.

Your app: the working baseline

Your app should meet this bar:

- The core user flow works from start to finish at least once, such as starting a game and finishing it.
- I can run it from your repo using your README instructions.
- You can explain how it works and what your spec asked for.

Anything beyond that is a bonus. If parts are broken, document them clearly instead of hiding them. Honest, well-documented failures earn more credit than undisclosed ones.

Testing (required categories, your format)

Your test log must cover each category below. Use whatever format works for you (table, markdown, spreadsheet, prose).

1. **Functional testing:** Does each feature in your spec behave as intended?

2. **Edge cases and negative testing:** Unexpected input, empty states, rapid clicking, refreshing mid-action.
3. **Usability testing:** At least one other person (a classmate or someone outside the class) uses your app without help. Record where they struggled.
4. **Security probing:** At least three probes, such as viewing page source, editing saved state or dev tools values, checking for exposed keys or answers, or trying odd inputs or files if your app has uploads. Only test your own app, or a classmate's with their permission.
5. **AI-assisted review:** Ask an AI tool to review your code as a security or QA tester, then record which findings you verified, which you rejected, and why.
 - The tool used to review should be different than what you used for most of your development work. Please mention which tool you used.

Every log entry needs at least: what you tested, how, what happened, and what you did about it (fixed, deferred, or won't fix). Include severity if it helps.

Analysis: what would it take to get this fully working?

In your write-up, address:

- **Status:** What works, what partly works, what doesn't.
- **Root causes:** Why the broken parts are broken (spec gaps, AI-generated code issues, your own decisions).
- **Path forward:** The specific steps, in priority order, to reach a fully working version, and roughly how much effort each takes.
- **Reflection:** What you'd do differently in your prompts, spec or testing approach.

Presentation (5-7 minutes plus discussion)

- **About 1 min:** What the app is and who it's for.
- **About 2-3 min:** Live demo (consider a backup recording or screenshots in case the live demo fails).
- **About 2 min:** Your most interesting testing finding and what you did about it.
- **About 1 min:** Gap analysis and next steps.
- Then class questions and discussion. Come ready to ask a classmate one question too.

Rough idea of how I will grade

I asked Claude to suggest a rubric for grading that will acknowledge that your app may not be perfect. Again, I want you to demonstrate that you've built and tested something, and are thinking critically about the value and challenges of AI-assisted development.

This is that rough overview; as I review your apps, the final breakdown **may** be different:

Component	Points
Working baseline and repo quality	25
Testing (coverage of all five categories, quality of the log)	30
Analysis and write-up	20
Presentation and discussion	15
Spec Kit process (artifacts included, spec matches what was built)	10

Rules and expectations

- **AI use is expected.** Include your key prompts or spec artifacts, and be ready to explain any code you present.
- **Security testing scope:** own app, or with other students' permission.
- **No late or incomplete work:** submit what you have and document what's missing.